# Unit 4 – Blockchain and Decentralized Applications

## Lab 5 Manual

**Objective:**

In this lab, students will learn and implement inheritance and mappings across inherited contracts for data management.

By the end of this lab, students will be able to:

- Reuse code by allowing contracts to derive properties and functions from other contracts using inheritance.
- Override parent contract functions in derived contracts and understand how to customize behaviour.
- Use mappings effectively within inherited contracts to manage data structures across different contract layers.
- Call parent contract functions explicitly using the super keyword and understand its significance.
- Understand the internal and external function modifiers and learn when and how to use them effectively in inheritance scenarios.

5.1 Basics of inheritance

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Parent {
    string public greeting = "Hello from Parent";

    ftrace | funcSig
    function sayHello() public view virtual returns (string memory) {
        return greeting;
    }
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Child is Parent {
    // Override the parent function
    ftrace | funcSig
    function sayHello() public view override returns (string memory) {
        return "Hello from Child";
    }
}
```

5.2 Mapping with Inheritance

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Parent {
    mapping(uint => string) public records;

    ftrace | funcSig
    function setRecord(uint id, string memory data) public {
        records[id] = data;
    }
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Child is Parent {
    ftrace | funcSig
    function getRecord(uint id) public view returns (string memory) {
        return records[id];
    }
}
```

5.3 Accessing Parent State Variables

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Parent {
    uint public parentValue = 100;
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Child is Parent {
    ftrace | funcSig
    function getParentValue() public view returns (uint) {
        return parentValue;
    }
}
```

5.4 The use of super keyword: The super keyword is useful in cases of multiple inheritance, where the child contract needs to ensure that the correct parent contract's function is called.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Parent {
    ftrace | funcSig
    function greet() public pure virtual returns (string memory) {
        return "Greetings from Parent";
    }
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Child is Parent {
    ftrace | funcSig
    function greet() public pure override returns (string memory) {
        return string(abi.encodePacked(super.greet(), " and Child"));
    }
}
```

5.5 Super keyword

```sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Grandparent {
    ftrace | funcSig
    function identify() public pure virtual returns (string memory) {
        return "Grandparent";
    }
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Parent1 is Grandparent {
    ftrace | funcSig
    function identify() public pure virtual override returns (string memory) {
        return "Parent1";
    }
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Parent2 is Grandparent {
    ftrace | funcSig
    function identify() public pure virtual override returns (string memory) {
        return "Parent2";
    }
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Child is Parent1, Parent2 {
    ftrace | funcSig
    function identify() public pure override(Parent1, Parent2) returns (string memory) {
        return string(abi.encodePacked(super.identify(), " -> Child"));
    }
}
```

5.6 Internal vs. External Functions: *internal* functions can only be accessed within the contract or derived contracts, whereas *external* functions are callable only from outside the contract.

```sol
uml | draw.io | funcSigs | report | graph (this) | graph | inheritance | parse | flatten
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Parent {
    uint internal internalValue = 42;

    ftrace | funcSig
    function externalFunction() external pure returns (string memory) {
        return "This is an external function";
    }

    ftrace | funcSig
    function internalFunction() internal view returns (uint) {
        return internalValue;
    }
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Child is Parent {
    ftrace | funcSig
    function useInternalFunction() public view returns (uint) {
        return internalFunction();
    }
}
```

## 5.7 Internal vs. External Functions

```
uml | draw.io | funcSigs | report | graph (this) | graph | inheritance | parse | flatten
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Parent {
    // Internal function
    ftrace | funcSig
    function _calculateInternal(uint a1, uint b1) internal pure returns (uint) {
        return a1 + b1;
    }

    // External function
    ftrace | funcSig
    function calculateExternal(uint a1, uint b1) external pure returns (uint) {
        return a1 * b1;
    }
}

UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Child is Parent {
    // Accessing the internal function from the parent
    ftrace | funcSig
    function useInternalCalculation(uint a1, uint b1) public pure returns (uint) {
        return _calculateInternal(a1, b1); // Can be accessed
    }

    // Attempting to use the external function directly within the contract
    // function useExternalCalculation(uint a, uint b) public pure returns (uint) {
    //     return calculateExternal(a, b); // Error: external functions can't be called internally
    // }
}
```

**Lab Task**

Develop a decentralized library management System:

**Concepts Used:**

- *Inheritance:* Base contract for library features and derived contracts for specific types of libraries (e.g., physical books, eBooks).
- *Mapping:* To manage book inventory or track borrower details.
- *Internal vs. External:* Internal functions for managing book data and external functions for user interactions.

**Details:**

- Create a base contract for adding and tracking books.
- Use derived contracts for different types of libraries (e.g., university library, community library).
- Add functionality for issuing and returning books.
- Track users who borrowed books using mappings.

**Some More:**

A. Inheritance

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
UnitTest stub | dependencies | uml | funcSigs | draw.io
contract A{
    ftrace | funcSig
    function add(uint a,uint b) public virtual pure returns(uint){
        uint c= a+b;
        return c;}
    ftrace | funcSig
    function sub(uint a,uint b) public pure returns(uint){
    require(a>b,"a is less then b");
        uint c= a-b;
        return c;
    }
}
UnitTest stub | dependencies | uml | funcSigs | draw.io
contract B {
    ftrace | funcSig
    function add(uint a,uint b) public virtual   pure  returns(uint) {
        uint c= a+b;
        return c;}
    ftrace | funcSig
    function divide(uint a,uint b) public pure returns(uint){
        require(a>b,"a is less then b");
        uint c= a/b;
        return c;}
}
UnitTest stub | dependencies | uml | funcSigs | draw.io
contract Inheritence is A, B{   //if we call parent class function
    ftrace | funcSig
    function add(uint a,uint b) public override(A,B)  pure returns(uint){
    require(a>b,"a is less then b");
        uint c= a+b;
        return c;}
}
```

B. We does not need a object of parent class, we directly access the public and internal visible functions in the child contract. Note that external function only usable from other contracts not in child nor in that contract which is declared.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
UnitTest stub | dependencies | uml | funcSigs | draw.io
contract A{      //external function visibility
    ftrace | funcSig
    function add(uint a↑,uint b↑) external pure  returns(uint){
        uint c= a↑+b↑;
        return c;
        }
    ftrace | funcSig
    function MainMultiplication(uint a↑,uint b↑) internal pure returns(uint){
        uint c=a↑*b↑;
        return c;
        }
    ftrace | funcSig
    function TakeInternalFunction(uint a↑,uint b↑) public pure returns(uint){
      uint c=MainMultiplication(a↑,b↑);
      return c;
      }
}
UnitTest stub | dependencies | uml | funcSigs | draw.io
contract B {
    A newa= new A();
    ftrace | funcSig
    function add(uint a↑,uint b↑)public view  returns(uint){
        uint c= newa.add(a↑, b↑);
        return c;
    }
}
UnitTest stub | dependencies | uml | funcSigs | draw.io
contract derivateA is A{
ftrace | funcSig
function childMultiplication(uint a↑,uint b↑)public pure  returns(uint) {

        uint c= MainMultiplication(a↑,b↑);
        return c;
    }
}
```